Lista de Exercícios n° 9

Capítulo 3 - Linguagem de Programação Haskell

1. Sem utilizar as funções pré-definidas da linguagem, implemente funções Haskell para o tratamento de listas. Nas operações que envolvem alterações nos elementos ou na estrutura de uma lista, a função deverá produzir uma nova lista.

   (a) Calcule o somatório de todos os elementos de uma lista.

   (b) Retorne o n-ésimo elemento de uma lista, considerando que a primeira posição possui índice zero.

   (c) Acrescente um elemento à primeira posição.

   (d) Acrescente um elemento à última posição.

   (e) Acrescente um elemento à n-ésima posição.

   (f) Acrescente um elemento a uma lista ordenada, mantendo a lista resultante também ordenada.

   (g) Retorne uma lista sem a primeira ocorrência de um determinado elemento.

   (h) Retorne uma lista sem nenhuma ocorrência de um determinado elemento.

   (i) Conte quantas vezes um determinado elemento ocorre na lista.

   (j) Retorne uma lista na qual todas as ocorrências de um determinado elemento sejam substituídas por outro elemento.

   (k) Encontre o maior elemento de uma lista não vazia.

   (l) Retorne os elementos da lista em ordem inversa.

2. Considere as funções a seguir, definidas utilizando compreensão de listas. Reescreva cada uma delas sem utilizar compreensão de listas, empregando recursão e casamento de padrões.

   (a)
   ```
   pares :: [Int] -> [Int]
   pares lista = [x | x <- lista, even x]
   ```

   (b)
   ```
   quadrados :: [Int] -> [Int]
   quadrados lista = [x * x | x <- lista]
   ```

   (c)
   ```
   quadradosPares :: [Int] -> [Int]
   quadradosPares lista =
       [x * x | x <- lista, even x]
   ```

   (d)
   ```
   maiores :: Int -> [Int] -> [Int]
   maiores n lista =
       [x | x <- lista, x > n]
   ```

3. As funções a seguir foram definidas utilizando recursão e casamento de padrões. Reescreva cada uma delas utilizando compreensão de listas.

   (a)
   ```
   dobro :: [Int] -> [Int]
   dobro [] = []
   dobro (x:xs) = 2 * x : dobro xs
   ```

(b)
```
positivos :: [Int] -> [Int]
positivos [] = []
positivos (x:xs)
    | x > 0     = x : positivos xs
    | otherwise = positivos xs
```

(c)
```
somaUmPares :: [Int] -> [Int]
somaUmPares [] = []
somaUmPares (x:xs)
    | even x    = (x + 1) : somaUmPares xs
    | otherwise = somaUmPares xs
```

4. Considere uma matriz de inteiros representada por uma lista de listas. Implemente, utilizando compreensão de listas, funções que:

   (a) obtenham os elementos pares de uma determinada coluna;

   (b) produzam uma lista contendo a soma dos elementos de cada linha da matriz.